

Juego de mosaicos

Barchin y Charos están jugando un juego en una cuadrícula de $2N \times 2M$ celdas cuadradas unitarias. Las filas están numeradas 0 al $2N - 1$ de arriba a abajo, y las columnas están numeradas 0 al $2M - 1$ de izquierda a derecha. Para $0 \leq i < 2N$ y $0 \leq j < 2M$, denotamos la celda en la fila i y la columna j por (i, j) .

Barchin le da a Charos $N \cdot M$ **bloques**, uno por uno. Cada bloque es un cuadrado de 2×2 que consta de cuatro 1×1 **mosaicos**. Barchin ha coloreado cada mosaico del bloque de negro o blanco, garantizando que **al menos un mosaico sea blanco**.

Charos debe colocar cada bloque en la cuadrícula **inmediatamente después de recibirlo**, sin saber qué bloques recibirá más adelante. Los bloques no se pueden rotar. Cada bloque debe colocarse completamente dentro de la cuadrícula, cubriendo exactamente cuatro celdas. Además, la casilla superior izquierda de cada bloque debe cubrir una celda cuyas coordenadas de fila y columna sean pares. Cada celda de la cuadrícula puede estar cubierta por un máximo de un bloque.

Barchin gana el juego si, luego de que cualquier bloque esté colocado, existe un cuadrado de celdas de 2×2 cubierto por cuatro mosaicos negros. Formalmente, si las celdas (a, b) , $(a + 1, b)$, $(a, b + 1)$ y $(a + 1, b + 1)$ están todas cubiertas por mosaicos negros para algún $0 \leq a < 2N - 1$ y $0 \leq b < 2M - 1$, entonces Barchin gana. Los índices a y b no tienen por qué ser pares.

Charos gana si coloca todos los bloques $N \cdot M$ sin que Barchin gane nunca. Tenga en cuenta que al colocar $N \cdot M$ bloques se cubrirá completamente la cuadrícula.

Tu tarea consiste en implementar una estrategia para que Charos gane la partida. Se puede demostrar que, bajo las restricciones dadas, Charos siempre puede colocar los bloques de manera que garantice la victoria, independientemente del color de los bloques que reciba posteriormente.

Detalles de la implementación

Debes implementar dos procedimientos:

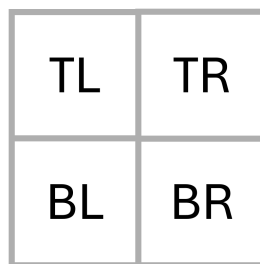
```
void init(int N, int M)
```

- N : la mitad del número de filas de la cuadrícula.

- M : la mitad del número de columnas de la cuadrícula.
- El procedimiento se llama exactamente una vez por cada caso de prueba, al inicio de la ejecución de tu programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : los colores de los mosaicos superior izquierdo, superior derecho, inferior izquierdo e inferior derecho del bloque actual, respectivamente, tal y como se muestra en la figura siguiente. Cada valor es 0 (blanco) o 1 (negro).
- Este procedimiento se invoca exactamente $N \cdot M$ veces por cada caso de prueba, tras la llamada inicial `ainit`.



Este procedimiento debe devolver un par de números enteros (i, j) , donde i es la coordenada de fila y j es la coordenada de columna de la celda en la que debe colocarse el mosaico superior izquierdo de este bloque. Tanto i como j deben ser pares, y la región 2×2 cubierta por el bloque no debe solaparse con ningún bloque colocado anteriormente.

Si `receive_block` devuelve en algún momento un par que no cumpla estos requisitos, o si tras colocar el bloque un cuadrado de celdas de 2×2 queda completamente cubierto por mosaicos negros, el grader terminará inmediatamente tu programa y el veredicto para el caso de prueba será `Output isn't correct`.

El comportamiento del grader **no es adaptativo**. Esto significa que la secuencia de bloques que Barchin le da a Charos está fijada antes de que se llame `ainit`.

Restricciones

Para cada bloque, sea S el número de mosaicos negros entre sus cuatro fichas. Es decir, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ por cada bloque.

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	6	$S = 1$ por cada bloque, y $N = 2$.
2	16	$S = 3$ por cada bloque. $N = M$, N es par, y cada una de las cuatro posibles coloraciones de bloques aparece exactamente $\frac{N^2}{4}$ veces.
3	10	$S = 1$ por cada bloque.
4	29	$S \leq 2$ por cada bloque.
5	39	Sin restricciones adicionales.

Ejemplo

Consideremos un juego con $N = 1$ y $M = 2$, de modo que la cuadrícula tiene 2 filas y 4 columnas. El grader llama primero:

```
init(1, 2)
```

Inicialmente, todas las celdas están vacías. La cuadrícula se ve así:

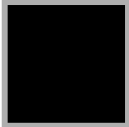
	0	1	2	3
0				
1				

Hay $N \cdot M = 2$ bloques para colocar. Supongamos que Barchin da un bloque con tres mosaicos negros y un mosaico blanco en la esquina superior derecha. El clasificador dice:

```
receive_block(1, 0, 1, 1)
```

Charos decide colocar este bloque en el lado izquierdo de la cuadrícula devolviendo $(0, 0)$.

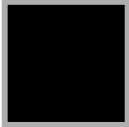
La cuadrícula ahora se ve así:

	0	1	2	3
0				
1				

A continuación, Barchin coloca otro bloque con tres mosaicos negros y una mosaico blanco en la esquina superior izquierda:

```
receive_block(0, 1, 1, 1)
```

La única celda restante con fila par y columna par que puede servir como esquina superior izquierda de un bloque de 2×2 es $(0, 2)$, por lo que Charos devuelve $(0, 2)$. La cuadrícula termina así:

	0	1	2	3
0				
1				

Ningún cuadrado 2×2 está completamente cubierto por mosaicos negros, por lo que Charos ha colocado todos los bloques sin que Barchin haya ganado. Charos gana la partida.

Grader de Ejemplo

Formato de entrada:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato de salida:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Aquí, $R[k]$ y $C[k]$ son el par de enteros devueltos por la llamada k a `receive_block`.